

ASSESSMENT TOOL-CO3-AT1

EXPERIMENT

Name: S. Bhuvanesh

Reg No: 192521004

Question 5: Analyze the trade-offs between Merge Sort and Quick Sort by comparing their execution time, space complexity, and stability through experimental implementation. Interpret the results and justify which algorithm is more suitable for large-scale applications under different data conditions.

1. Algorithm

Merge Sort

1. Divide the array into two halves.
2. Sort both halves recursively.
3. Merge the sorted halves.
4. Display execution time.

Quick Sort

1. Select a pivot.
2. Partition the array around the pivot.
3. Sort both parts recursively.
4. Display execution time.

2. Pseudocode

Merge Sort

MergeSort(A, low, high)

if low < high

 mid = (low + high)/2

 MergeSort(A, low, mid)

 MergeSort(A, mid+1, high)

 Merge(A, low, mid, high)

Quick Sort

QuickSort(A, low, high)

if low < high

 p = Partition(A, low, high)

 QuickSort(A, low, p-1)

 QuickSort(A, p+1, high)

3. C Program

```
#include <stdio.h>
```

```
#include <time.h>
```

```
void merge(int a[],int l,int m,int r){
```

```
    int i=l,j=m+1,k=0,t[50];
```

```
    while(i<=m&& j<=r)t[k++]=(a[i]<a[j])?a[i++]:a[j++];
```

```
    while(i<=m)t[k++]=a[i++];
```

```

    while(j<=r)t[k++]=a[j++];

    for(i=l,k=0;i<=r;i++)a[i]=t[k++];
}

void ms(int a[],int l,int r){
    if(l<r){
        int m=(l+r)/2;

        ms(a,l,m); ms(a,m+1,r); merge(a,l,m,r);
    }
}

int part(int a[],int l,int r){
    int p=a[r],i=l-1,t;

    for(int j=l;j<r;j++)

        if(a[j]<=p){t=a[++i];a[i]=a[j];a[j]=t;}

    t=a[i+1];a[i+1]=a[r];a[r]=t;

    return i+1;
}

void qs(int a[],int l,int r){
    if(l<r){
        int p=part(a,l,r);

        qs(a,l,p-1); qs(a,p+1,r);
    }
}

int main(){
    int n,a[50],b[50];

```

```

clock_t s,e;

printf("Enter n: ");

scanf("%d",&n);

printf("Enter elements: ");

for(int i=0;i<n;i++) scanf("%d",&a[i]),b[i]=a[i];

s=clock(); ms(a,0,n-1); e=clock();

printf("Merge Sort Time = %f\n",(double)(e-s)/CLOCKS_PER_SEC);

s=clock(); qs(b,0,n-1); e=clock();

printf("Quick Sort Time = %f\n",(double)(e-s)/CLOCKS_PER_SEC);

printf("Sorted: ");

for(int i=0;i<n;i++) printf("%d ",a[i]);

}

```

4. Sample Input

Enter n: 8

Enter elements: 38 27 43 3 9 82 10 5

5. Manual Process

Merge Sort:

38 27 43 3 9 82 10 5

→ 27 38 3 43 9 82 5 10

→ 3 27 38 43 5 9 10 82

→ 3 5 9 10 27 38 43 82

Quick Sort:

Pivot = 5

→ 3 5 43 38 9 82 10 27

→ 3 5 9 10 27 38 43 82

program execution:

```
main.py
1 import time
2
3 def merge_sort(a):
4     if len(a) <= 1: return a
5     m = len(a)//2
6     L, R = merge_sort(a[:m]), merge_sort(a[m:])
7     return sorted(L + R)
8
9 def quick_sort(a):
10    if len(a) <= 1: return a
11    p = a[-1]
12    return quick_sort([x for x in a[:-1] if x <= p]) + \
13           [p] + quick_sort([x for x in a[:-1] if x > p])
14
15 a = list(map(int, input("Enter elements: ").split()))
16
17 s = time.perf_counter()
18 m = merge_sort(a)
19 mt = time.perf_counter() - s
20
21 s = time.perf_counter()
22 q = quick_sort(a)
23 qt = time.perf_counter() - s
24
25 print("Merge Sort:", m)
26 print("Quick Sort:", q)
27 print("Merge Time:", mt)
28 print("Quick Time:", qt)
29 print("Merge Sort: O(n log n), Stable, O(n) space")
30 print("Quick Sort: O(n log n) average, Not Stable, O(log n) average space")
31
```

Output:

```
Enter elements: 50 23 6 45 9 10 75
Merge Sort: [6, 9, 10, 23, 45, 50, 75]
Quick Sort: [6, 9, 10, 23, 45, 50, 75]
Merge Time: 3.362299958098447e-05
Quick Time: 6.184100038808538e-05
Merge Sort: O(n log n), Stable, O(n) space
Quick Sort: O(n log n) average, Not Stable, O(log n) average space
```